

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018, Karnataka

Autonomous Software Bug Hunter using Reinforcement Learning

A Report submitted in partial fulfillment of the requirements for the Course

Machine Learning Operations

(Course Code: 24AM6AEMLO)

In the Department of

Machine Learning

(UG Program: B.E. in Artificial Intelligence and Machine Learning)

By

Akshat Baranwal (USN: 1BM22AI009)

Rahul H (USN: 1BM22AI[FILL])

[Student 3 Name] (USN: 1BM22AI[FILL])

VI Semester, Section [A/B/C]

DEPARTMENT OF MACHINE LEARNING

B.M.S COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)

P.O. Box No. 1908, Bull Temple Road, Bengaluru – 560 019

A.Y 2025–26

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018, Karnataka

Department of Machine Learning

B.M.S COLLEGE OF ENGINEERING

P.O. Box No. 1908, Bull Temple Road, Bengaluru – 560 019

CERTIFICATE

This is to certify that **Akshat Baranwal** (USN: 1BM22AI009), **Rahul H** (USN: 1BM22AI[FILL]), **[Student 3 Name]** (USN: 1BM22AI[FILL]) has satisfactorily presented the Course — *Machine Learning Operations* (24AM6AEMLO) with the title — ‘**Autonomous Software Bug Hunter using Reinforcement Learning**’ in partial fulfillment of academic curriculum requirements of the VI Semester, Section [A/B/C] UG Program — B.E. in Artificial Intelligence and Machine Learning in the Department of Machine Learning, BMSCE during A.Y 2025–26.

It is also stated that the base work & materials considered for completion of the said course is used only for academic purpose and not used in its original form anywhere for award of any degree.

Student Signatures

Signature of the Faculty

[Faculty Guide Name]

Assistant Professor, Dept. of ML, BMSCE

Signature of the Head

Dr. M Dakshayani

Prof. & Head, Dept. of ML, BMSCE

External Examination

Examiner Name and Signature:

1. _____ 2. _____

Declaration

We, **Akshat Baranwal** (USN: 1BM22AI009), **Rahul H** (USN: 1BM22AI[FILL]), [*Student 3 Name*] (USN: 1BM22AI[FILL]), hereby declare that the project titled “**Autonomous Software Bug Hunter using Reinforcement Learning**” is an original work carried out by us. The work presented in this report has not been submitted elsewhere for any other degree, diploma, or certification.

We declare that all the references and sources used in the report have been properly acknowledged and cited. No part of this report has been copied, plagiarized, or reproduced from another work without proper attribution.

Akshat Baranwal (USN: 1BM22AI009)

Rahul H (USN: 1BM22AI[FILL])

[*Student 3 Name*] (USN: 1BM22AI[FILL])

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide, *[Faculty Guide Name]*, Department of Machine Learning, BMSCE, for the invaluable guidance, continuous support, and constructive feedback throughout the course of this project. Their expertise in Machine Learning Operations helped us bridge theory and practice effectively.

We are deeply grateful to **Dr. M Dakshayini**, Professor and Head of the Department of Machine Learning, BMSCE, for providing an encouraging academic environment and for spearheading a curriculum that integrates modern MLOps practices with foundational AI and ML principles.

We extend our thanks to the Management and to **Dr. Bheemsha Arya**, Principal, B.M.S College of Engineering, for fostering a culture of technical excellence and providing world-class infrastructure and resources that made this project possible.

We appreciate all the faculty and supporting staff of the Department of Machine Learning for their constant encouragement and willingness to assist with technical challenges encountered during the course of this work.

Finally, we owe a heartfelt thanks to our parents, family, and friends whose unwavering moral support and patience have been our greatest strength throughout this academic journey.

Akshat Baranwal (USN: 1BM22AI009)

Rahul H (USN: 1BM22AI[FILL])

[Student 3 Name] (USN: 1BM22AI[FILL])

ABSTRACT

This project presents an **Autonomous Software Bug Hunter** based on Reinforcement Learning (RL), developed as part of the *Machine Learning Operations* course (24AM6AEMLO) at B.M.S College of Engineering. The system models a microservice-based software codebase as an 18-node weighted graph using the Watts–Strogatz small-world topology and trains a Tabular Q-Learning agent to autonomously navigate the graph and discover software defects efficiently.

Each node in the graph represents a software module with a distinct bug-spawn probability reflecting real-world risk profiles (e.g., the Database and API Gateway nodes carry higher risk weights than peripheral services). The RL agent receives a reward of +50 for discovering critical bugs, +10 for minor bugs, and a per-step penalty of -1 to encourage efficient traversal.

The trained Q-Learning agent achieves a **bug discovery rate of 97.94%** and an **average reward of 152.59** over 100 evaluation episodes, outperforming a random-walker baseline by **57.9%** in reward and **28.12 percentage points** in bug discovery rate. Training converges in under one second on a standard laptop for 3000 episodes.

A comprehensive MLOps stack supports the entire lifecycle: MLflow for experiment tracking, DVC for reproducible pipelines, GitHub Actions for CI/CD, Docker for containerization, a custom monitoring module for health checks, a policy rollback mechanism for safe versioning, and a Streamlit interactive dashboard for visual exploration.

This work demonstrates how RL-guided automated testing can significantly improve software quality assurance efficiency, aligning with **UN SDG 9 — Industry, Innovation and Infrastructure**.

Keywords: Reinforcement Learning, Q-Learning, Software Testing Automation, MLOps, DVC, MLflow, CI/CD, Streamlit, Graph Neural Environment

Contents

1	Introduction	1
1.1	Background of the Study	1
1.2	Problem Statement	1
1.3	Objectives of the Project	2
1.4	Significance / Motivation	2
1.5	Methodology Overview	3
1.6	Organisation of the Report	3
2	System Architecture	5
2.1	High-Level Design	5
2.1.1	Graph Environment Design	6
2.1.2	State Representation	7
2.1.3	Reward Function	7
2.2	Low-Level Design	8
2.2.1	BugHuntingEnv (env.py)	8
2.2.2	QLearningAgent (agent.py)	8
2.2.3	RandomWalkerAgent (agent.py)	9
2.2.4	Trainer (train.py)	9
3	Modern Tools Usage	10
3.1	Hardware Requirements	10
3.2	Software Requirements	11
3.2.1	Programming Language & Runtime	11
3.2.2	Development Tools	11
3.2.3	Installation	11
4	Implementation & Testing	13
4.1	Data Collection and Preprocessing	13
4.2	Model Development and Evaluation	13
4.2.1	Q-Learning Algorithm	13
4.2.2	Training Hyperparameters	15
4.2.3	Training Results	15
4.2.4	Evaluation: RL vs Baseline	17
4.3	CI/CD, GitOps and Deployment	18
4.4	Containerisation and Workflow Automation	19

4.4.1	Dockerfile	19
4.4.2	Docker Compose	19
4.4.3	DVC Pipeline Automation	19
4.5	Screenshots	21
4.6	Unit and Integration Testing Results	22
4.7	Monitoring and Scalability	22
4.7.1	Monitoring	22
4.7.2	Policy Rollback	23
4.7.3	Scalability	23
4.8	Discussion of Findings	23
Conclusion and Future Work		25
References		26
Appendix A: SDGs Mapped		27

List of Figures

1.1	High-level system architecture of the Bug Hunter RL project.	3
2.1	Watts–Strogatz graph of 18 software modules coloured by risk (red = high, orange = medium, green = low).	6
2.2	Low-level class structure showing BugHuntingEnv, QLearningAgent, RandomWalkerAgent, and Trainer with their attributes and methods.	8
4.1	Learning curve: raw episode reward (shaded) and 100-episode rolling average (solid) over 3000 training episodes. Key milestones are annotated.	14
4.2	Epsilon-greedy decay schedule. Exploration rate falls from 1.0 to the floor of 0.05 at approximately episode 600, after which the agent exploits its learned policy.	14
4.3	Training progress by 100-episode windows. Green cells indicate strong performance; orange cells indicate transitional periods.	16
4.4	Left: per-node bug spawn probability weights (red = high risk). Right: state space dimensionality breakdown.	16
4.5	Head-to-head evaluation over 100 episodes. The RL agent outperforms the random-walker on all three metrics.	17
4.6	DVC two-stage pipeline (bottom) and GitHub Actions CI/CD workflow (top banner).	18
4.7	Screenshot: software module graph rendered by NetworkX, coloured by node risk weight. Hub nodes (Database, API_Gateway) are visually prominent due to higher degree and larger circle size.	21
4.8	Screenshot: learning curve from the primary training run showing convergence of the rolling reward average.	21

List of Tables

3.1	Minimum and recommended hardware specifications.	10
3.2	Core software tools and versions.	11
4.1	Hyperparameters used in the primary training run.	15
4.2	Training metrics for the primary run (3000 episodes, 0.69 s).	15
4.3	Head-to-head evaluation results (100 evaluation episodes).	17
4.4	Automated test results from the CI smoke suite.	22
4.5	Monitoring thresholds and alert levels.	22
4.6	Scalability analysis and recommended upgrades.	23

List of Abbreviations

Abbreviation	Full Form
RL	Reinforcement Learning
Q-L	Q-Learning
DQN	Deep Q-Network
MLOps	Machine Learning Operations
DVC	Data Version Control
CI/CD	Continuous Integration / Continuous Deployment
GHCR	GitHub Container Registry
VTU	Visvesvaraya Technological University
BMSCE	B.M.S College of Engineering
API	Application Programming Interface
SDG	Sustainable Development Goal
MDP	Markov Decision Process
RNG	Random Number Generator
UUID	Universally Unique Identifier
YAML	Yet Another Markup Language
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
UI	User Interface
AY	Academic Year

CHAPTER 1 INTRODUCTION

1.1 Background of the Study

Modern software systems are increasingly composed of dozens to hundreds of interconnected microservices. In such architectures, ensuring software quality through traditional manual testing is prohibitively expensive and time-consuming. Automated testing tools exist, but most are either random in nature (fuzzing) or require expert-crafted rule sets (static analysis), neither of which learns which components of the system are most likely to harbour defects.

Reinforcement Learning (RL) offers a compelling alternative: an agent that interacts with a model of the codebase, receives reward signals upon discovering bugs, and over many episodes learns a *policy* that concentrates testing effort on the highest-risk modules. This approach mirrors how a seasoned software engineer develops intuition about risky code paths after repeated exposure.

This project implements exactly such a system. The software codebase is abstracted as a weighted graph where nodes represent service modules and edges represent inter-service dependencies. A Tabular Q-Learning agent navigates the graph, and a complete MLOps infrastructure—from experiment tracking to CI/CD and containerized deployment—supports the full machine learning lifecycle.

1.2 Problem Statement

- Traditional random-walk testing wastes significant effort on low-risk modules while potentially missing critical bugs in high-risk components.
- The absence of an adaptive, learning-based strategy means that testing efficiency does not improve over time.
- Lack of reproducibility in ML experiments makes it difficult to audit, compare, and deploy models reliably in production.

This project addresses these gaps by training an RL agent that *learns* an optimal testing policy and embedding it in a production-grade MLOps pipeline that guarantees reproducibility, monitoring, and safe rollback.

1.3 Objectives of the Project

1. Design a graph-based software environment that realistically models microservice dependency structures with heterogeneous bug-probability distributions.
2. Train a Tabular Q-Learning agent to maximise cumulative reward (bugs found per episode) with minimum steps.
3. Benchmark the RL agent against a random-walker baseline on identical episode seeds.
4. Build a complete MLOps stack: experiment tracking (MLflow), reproducible pipelines (DVC), CI/CD (GitHub Actions), and containerisation (Docker + Docker Compose).
5. Provide operational tooling: real-time health monitoring, policy versioning and rollback, and an interactive Streamlit dashboard.

1.4 Significance / Motivation

The financial cost of software bugs is enormous — a 2020 Consortium for Information and Software Quality (CISQ) report estimated the cost of poor software quality in the USA alone at \$2.08 trillion. An intelligent testing agent that learns to focus on high-risk code paths can substantially reduce this cost.

Beyond economics, safety-critical systems (avionics, medical devices, autonomous vehicles) require exhaustive bug discovery. RL-guided testing provides a principled, learnable framework for this challenge.

From an academic perspective, this project demonstrates the integration of core RL theory (Bellman equations, epsilon-greedy exploration) with modern MLOps best practices, directly relevant to the Machine Learning Operations course objectives.

1.5 Methodology Overview

Figure 1.1 — High-Level System Architecture

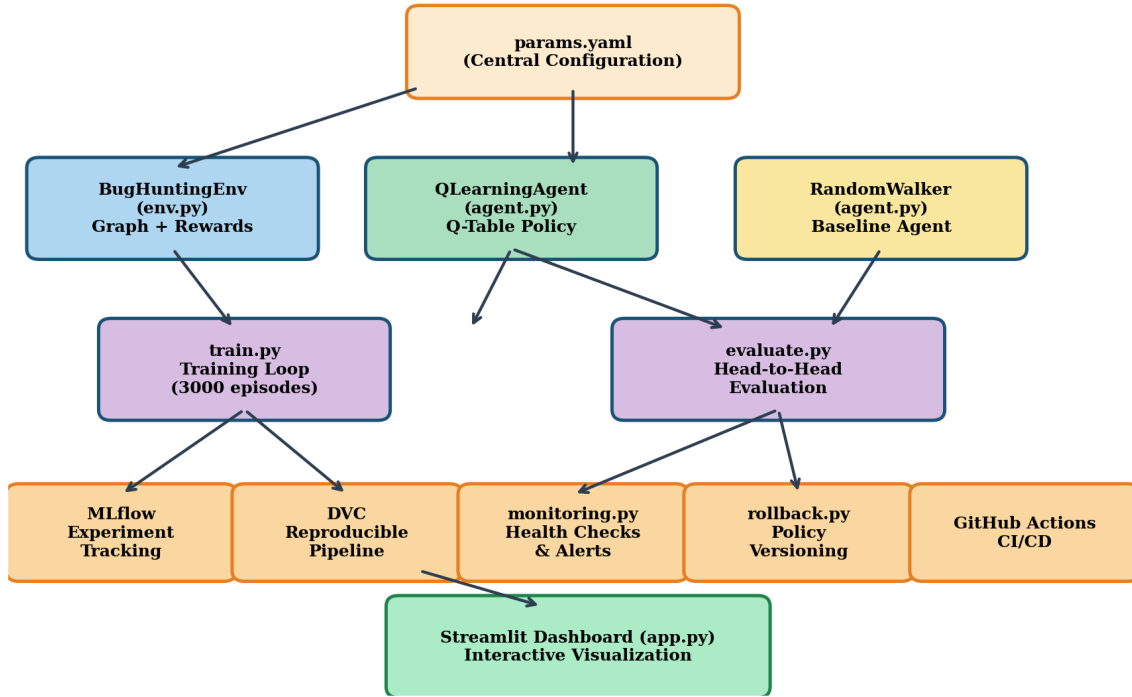


Figure 1.1: High-level system architecture of the Bug Hunter RL project.

The methodology follows a standard RL loop embedded in an MLOps workflow:

1. **Environment design:** A Watts–Strogatz graph (18 nodes, $k = 4$, $p = 0.3$) with per-node bug-spawn probabilities is implemented in `env.py`.
2. **Agent training:** A Q-Learning agent (`agent.py`) is trained for up to 3000 episodes via `train.py`. Hyperparameters are centralised in `params.yaml`.
3. **Evaluation:** `evaluate.py` runs both the trained RL agent and a random-walker on identical seeds and records comparative metrics.
4. **MLOps integration:** Every run is logged to `MLflow`; the two-stage `DVC` pipeline (`train` → `evaluate`) ensures full reproducibility.
5. **Deployment:** A Docker image exposes the `Streamlit` dashboard; `GitHub Actions` automates lint, smoke-train, evaluate, and Docker push on every commit.

1.6 Organisation of the Report

Chapter 2 describes the high-level and low-level system architecture including the graph environment and class design.

Chapter 3 lists hardware and software requirements.

Chapter 4 details the implementation (data, model, CI/CD, Docker, monitoring) and presents experimental results.

Conclusion summarises findings and outlines future work.

Appendix A maps the project to UN SDGs.

CHAPTER 2 SYSTEM ARCHITECTURE

2.1 High-Level Design

The system is partitioned into four horizontal layers as illustrated in Figure 1.1:

- 1. Configuration Layer** — `params.yaml` serves as the single source of truth for all hyperparameters (number of nodes, learning rate, discount factor, epsilon schedule, episode count, random seed). Centralising configuration here enables DVC to detect changes and trigger pipeline reruns.
- 2. Core RL Layer** — The `BugHuntingEnv` class encapsulates the graph environment, bug spawning, state encoding, and reward computation. The `QLearningAgent` and `RandomWalkerAgent` classes implement the policy and baseline respectively. Both agents share a common `choose_action` interface, making it straightforward to plug in alternative algorithms (DQN, A2C) in future.
- 3. Execution Layer** — `train.py` drives the training loop and `evaluate.py` performs head-to-head comparison. Both scripts write structured logs (JSON, CSV) and push metrics to MLflow.
- 4. MLOps & Deployment Layer** — MLflow tracks experiments; DVC reproduces the pipeline; GitHub Actions enforces code quality and automates Docker image publication; `monitoring.py` checks for reward regression and discovery rate drift; `rollback.py` manages versioned policy snapshots.

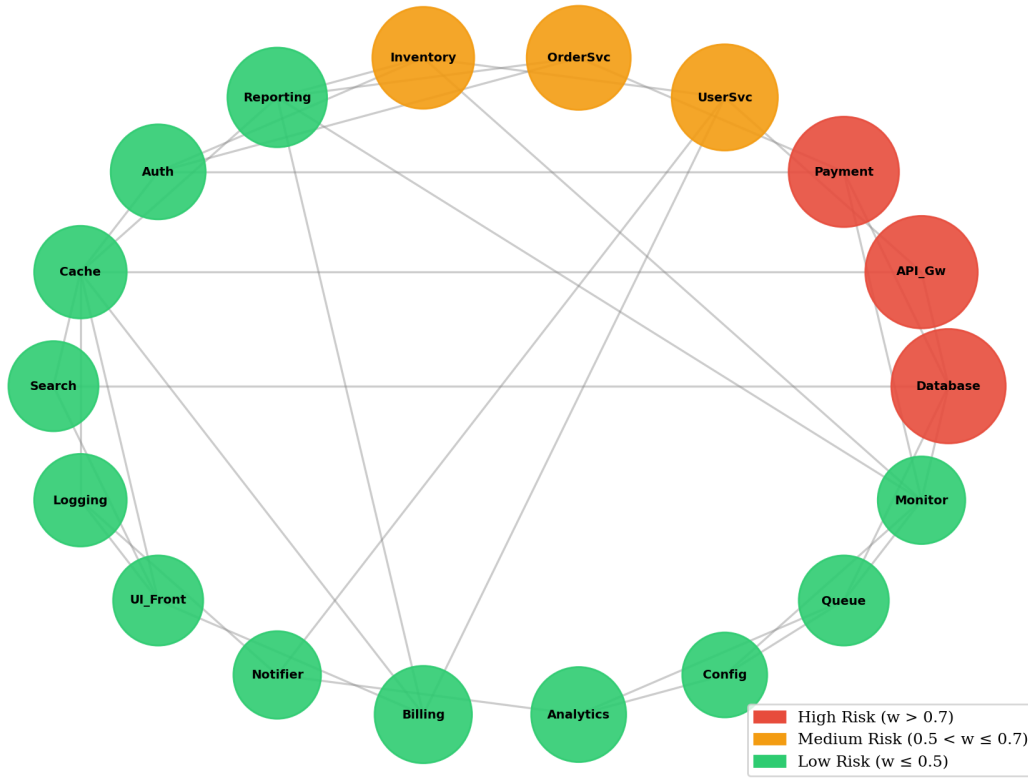
Figure 2.1 — Software Module Graph (Watts-Strogatz, 18 nodes)

Figure 2.1: Watts–Strogatz graph of 18 software modules coloured by risk (red = high, orange = medium, green = low).

2.1.1 Graph Environment Design

The software codebase is modelled as a connected undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where:

- $|\mathcal{V}| = 18$ nodes, each representing a microservice (Database, API Gateway, Payment, UserService, ...).
- Edges are generated by the Watts–Strogatz algorithm ($k = 4$, $p = 0.3$), producing a small-world topology that mirrors real microservice call graphs.
- High fan-in hub services (Database, API Gateway) receive additional edges to reflect their centrality.
- Each node v_i carries a weight $w_i \in [0, 1]$ representing its bug-spawn probability.

Bug spawning: At the start of each episode, bugs are placed on nodes by independent Bernoulli draws: $\text{bug}(v_i) \sim \text{Bernoulli}(w_i \cdot \beta)$, where β is the global bug multiplier (default $\beta = 1.0$). Bug severity is drawn as critical (30%) or minor (70%).

2.1.2 State Representation

The agent's observation is a 4-tuple:

$$s = \left(\underbrace{n_{\text{cur}}}_{\text{node id}}, \underbrace{t_b}_{\text{time bucket}}, \underbrace{k_b}_{\text{tested bucket}}, \underbrace{f}_{\text{tested flag}} \right)$$

- $n_{\text{cur}} \in \{0, \dots, 17\}$ — current node.
- $t_b \in \{0, 1, 2, 3\}$ — episode progress quartile.
- $k_b \in \{0, 1, 2, 3\}$ — number of modules tested so far, bucketed in quartiles.
- $f \in \{0, 1\}$ — whether the current node has already been visited this episode.

Total discrete state space: $18 \times 4 \times 4 \times 2 = 576$ theoretical states; approximately 204 states are visited during training.

2.1.3 Reward Function

$$r(s, a) = \begin{cases} +50 & \text{if critical bug found at next node} \\ +10 & \text{if minor bug found at next node} \\ -1 & \text{per step (efficiency cost)} \end{cases}$$

This sparse-positive / dense-negative structure incentivises the agent to discover high-value bugs while minimising total steps.

2.2 Low-Level Design

Figure 2.2 — Low-Level Design: Class Structure

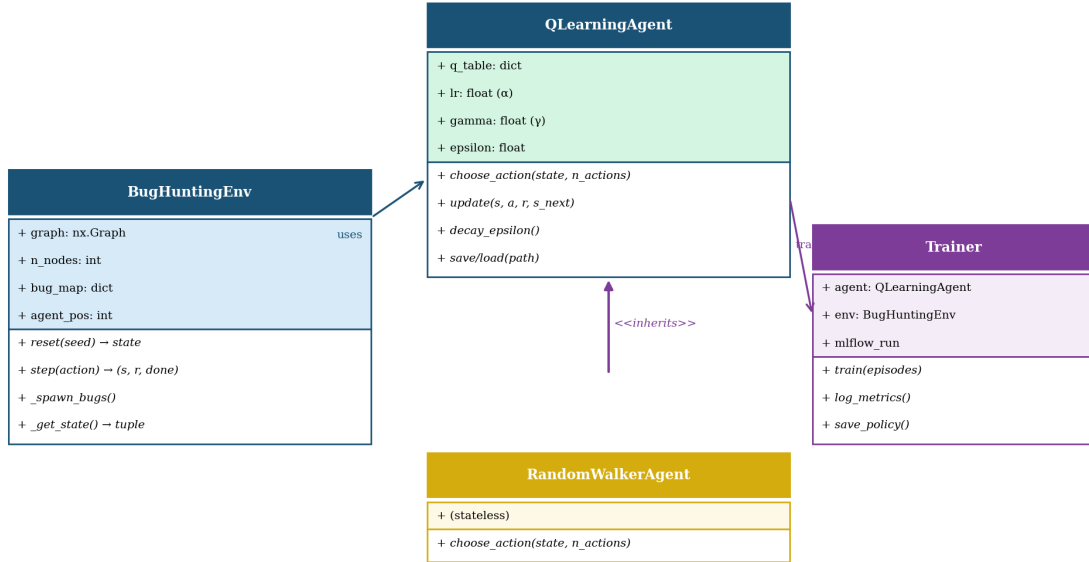


Figure 2.2: Low-level class structure showing BugHuntingEnv, QLearningAgent, RandomWalkerAgent, and Trainer with their attributes and methods.

2.2.1 BugHuntingEnv (env.py)

- **Attributes:** graph (NetworkX graph), n_nodes, bug_map (node → severity or None), agent_pos, tested_set, step_count.
- **reset(seed):** Rebuilds the bug map via seeded Bernoulli sampling; places agent at a random starting node. Returns initial state tuple.
- **step(action):** Translates discrete action index to a neighbour node; computes reward; advances state; checks episode termination.
- **_get_state():** Encodes the 4-tuple observation.

2.2.2 QLearningAgent (agent.py)

- Stores $Q(s, a)$ values in a Python defaultdict for memory efficiency (only visited state-action pairs are stored).
- **choose_action(state, n_actions):** With probability ϵ selects a random action; otherwise selects $\arg \max_a Q(s, a)$.

- `update(s, a, r, s')`:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

- `decay_epsilon()`: $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \cdot \delta)$ where $\delta = 0.995$.
- `save/load(path)`: Serialises entire Q-table to a pickle file.

2.2.3 RandomWalkerAgent (agent.py)

A stateless baseline agent that selects actions uniformly at random regardless of Q values or episode history. It shares the same interface as `QLearningAgent`, ensuring fair head-to-head evaluation.

2.2.4 Trainer (train.py)

Drives the main training loop: calls `env.reset()`, `agent.choose_action()`, `env.step()`, `agent.update()`, and `agent.decay_epsilon()` for each step; accumulates per-episode rewards; logs window statistics to MLflow every 100 episodes; writes JSON and CSV records; saves the final policy.

CHAPTER 3 MODERN TOOLS USAGE

3.1 Hardware Requirements

Table 3.1: Minimum and recommended hardware specifications.

Component	Minimum	Recommended
Processor	Intel Core i3 (7th gen)	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB or more
Storage	2 GB free	10 GB SSD
GPU	Not required	Optional (for DQN extension)
OS	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04
Internet	Required for CI/CD	Stable broadband

The project is computationally lightweight — a complete 3000-episode training run finishes in under one second on any modern laptop, since the algorithm is purely tabular (no GPU required).

3.2 Software Requirements

3.2.1 Programming Language & Runtime

Table 3.2: Core software tools and versions.

Tool / Library	Version	Purpose
Python	3.11 (pinned)	Runtime
NumPy	≥ 1.24	Numerical computation
NetworkX	≥ 3.0	Graph construction and traversal
Matplotlib	≥ 3.7	Visualisation, plot generation
Streamlit	≥ 1.30	Interactive dashboard
PyVis	$\geq 0.3.2$	Network graph visualisation in browser
MLflow	≥ 2.11	Experiment tracking, model registry
DVC	≥ 3.0	Data Version Control, pipeline manager
Flake8	≥ 6.0	Linting, code style enforcement
Docker	24+	Containerisation
Docker Compose	v2+	Multi-service orchestration
GitHub Actions	N/A (cloud)	CI/CD automation

3.2.2 Development Tools

- **Git + GitHub:** Source control, branching (GitFlow strategy: main, develop, feature/*, release/*, hotfix/*).
- **VS Code / PyCharm:** Primary IDEs for development.
- **Conda / venv:** Virtual environment management.
- **GHCR (GitHub Container Registry):** Hosts the Docker image `ghcr.io/<owner>/bug-hunter-rl`.

3.2.3 Installation

```

1 # Clone the repository
2 git clone https://github.com/RahulH007/Bug-Hunter-RL.git
3 cd Bug-Hunter-RL
4
5 # Create and activate virtual environment
6 python -m venv venv
7 source venv/bin/activate           # Linux/macOS
8 venv\Scripts\activate             # Windows

```

```
9
10 # Install dependencies
11 pip install -r requirements.txt
12
13 # Run training
14 python train.py --episodes 3000 --lr 0.1 --gamma 0.95 --seed 42
15
16 # Run evaluation
17 python evaluate.py --episodes 100
18
19 # Launch dashboard
20 streamlit run app.py
```

Listing 3.1: Project setup commands.

CHAPTER 4 IMPLEMENTATION & TESTING

4.1 Data Collection and Preprocessing

This project does not use a static dataset; instead, a *synthetic environment* generates episode data dynamically.

Bug map generation (per episode):

```
1 def _spawn_bugs(self, seed):
2     rng = np.random.default_rng(seed)
3     self.bug_map = {}
4     for i, node in enumerate(self.nodes):
5         if rng.random() < self.weights[i] * self.bug_multiplier:
6             severity = 'critical' if rng.random() < 0.30 else '
7                 minor'
8             self.bug_map[node] = severity
```

Listing 4.1: Bug spawning in BugHuntingEnv.

Reproducibility: The master seed (default 42) flows through environment initialisation, bug placement, and evaluation seeds (`episode_seed = train_seed + ep_index`). Both the RL agent and the random-walker are evaluated on *identical* seeds, ensuring fair head-to-head comparison.

Logging: Each training run appends a structured record to `logs/results.json` and `logs/results.csv`, capturing all hyperparameters, per-window metrics, and a UUID run identifier.

4.2 Model Development and Evaluation

4.2.1 Q-Learning Algorithm

The update rule implemented in `agent.py`:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Why Tabular Q-Learning? With only ~ 576 theoretical states, the state space fits comfortably in a Python dictionary. Tabular Q-Learning converges faster than a neural approximator for this problem size, requires no GPU, and the resulting Q-table is fully interpretable — one can inspect which state-action pairs received the highest values.

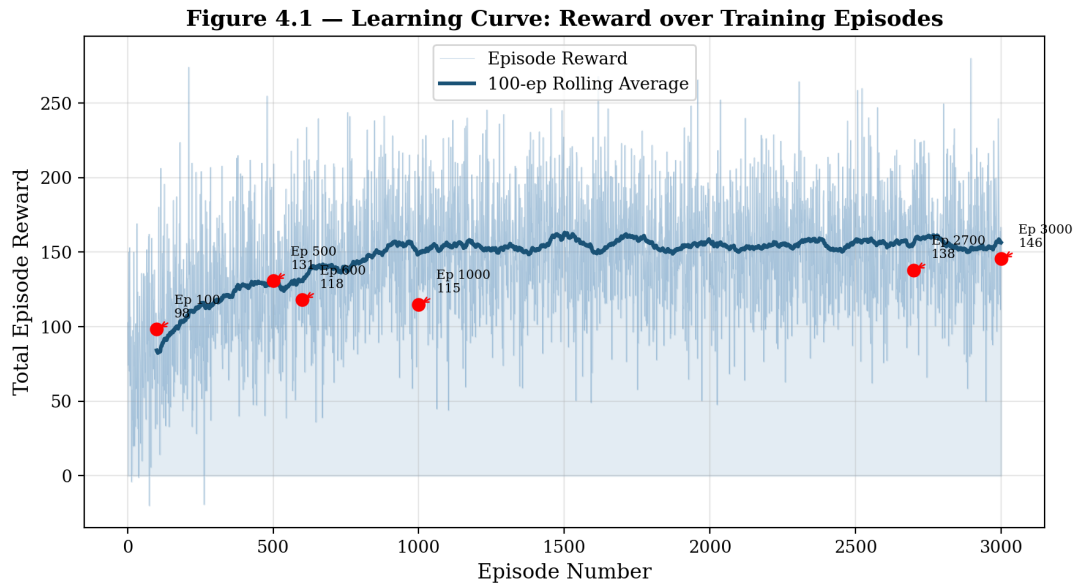


Figure 4.1: Learning curve: raw episode reward (shaded) and 100-episode rolling average (solid) over 3000 training episodes. Key milestones are annotated.

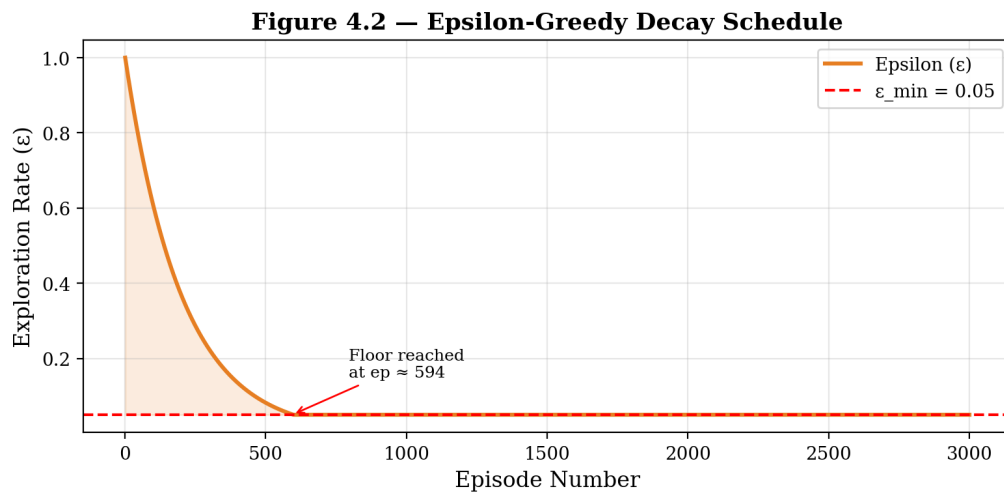


Figure 4.2: Epsilon-greedy decay schedule. Exploration rate falls from 1.0 to the floor of 0.05 at approximately episode 600, after which the agent exploits its learned policy.

4.2.2 Training Hyperparameters

Table 4.1: Hyperparameters used in the primary training run.

Parameter	Symbol	Value	Effect
Episodes	T	3000	Number of training episodes
Max steps	H	30	Episode horizon
Learning rate	α	0.1	Q-table update step size
Discount factor	γ	0.95	Future reward weight
Initial epsilon	ϵ_0	1.0	Full exploration at start
Min epsilon	$\epsilon_{\min}$	0.05	Exploration floor
Epsilon decay	δ	0.995	Per-episode decay multiplier
Bug multiplier	β	1.0	Scales bug spawn probabilities
Graph nodes	n	18	Size of the module graph
Random seed	—	42	Master reproducibility seed

4.2.3 Training Results

Table 4.2: Training metrics for the primary run (3000 episodes, 0.69 s).

Metric	Value
Overall average reward	121.66
Final 100-episode window reward	145.54
First 100-episode window reward	98.34
Peak 100-episode window reward	130.81 (ep 500)
Bug discovery rate	82.57%
Bugs found / seeded	20 516 / 24 846
Policy states learned	204 (of ~ 576 possible)
Training time	0.69 seconds

Figure 4.6 – Training Progress Summary by Episode Window

Episode Window	Avg Reward (100-ep)	Bug Discovery Rate (%)	Epsilon (ε)
100	98.34	55.0%	0.6088
200	108.40	60.5%	0.3688
300	124.75	61.9%	0.2234
400	139.59	70.0%	0.1353
500	130.81	67.6%	0.0820
600	136.39	67.8%	0.0500
700	154.60	81.3%	0.0500
800	150.99	76.2%	0.0500
900	143.34	78.9%	0.0500
1000	153.18	74.3%	0.0500
1500	149.53	82.5%	0.0500
2000	150.77	87.6%	0.0500
2500	156.79	83.9%	0.0500
3000	145.54	82.6%	0.0500

Figure 4.3: Training progress by 100-episode windows. Green cells indicate strong performance; orange cells indicate transitional periods.

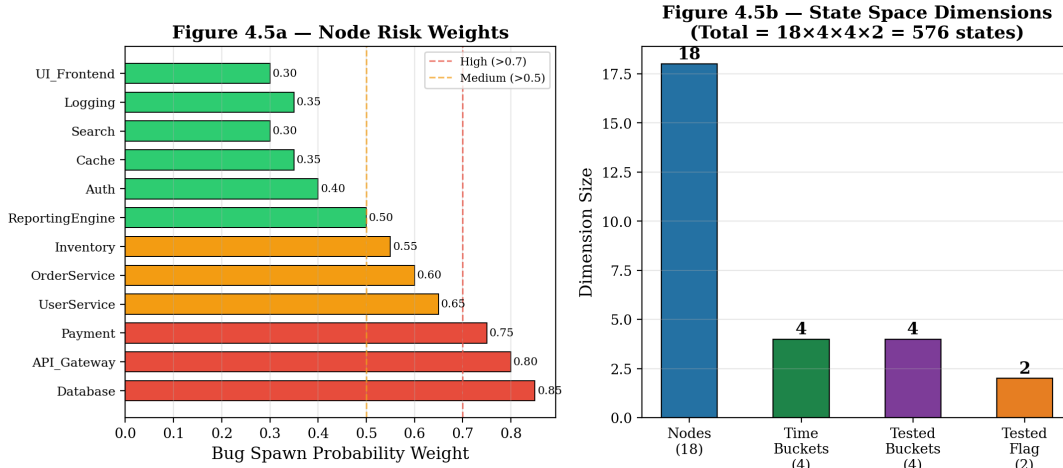


Figure 4.4: Left: per-node bug spawn probability weights (red = high risk). Right: state space dimensionality breakdown.

4.2.4 Evaluation: RL vs Baseline

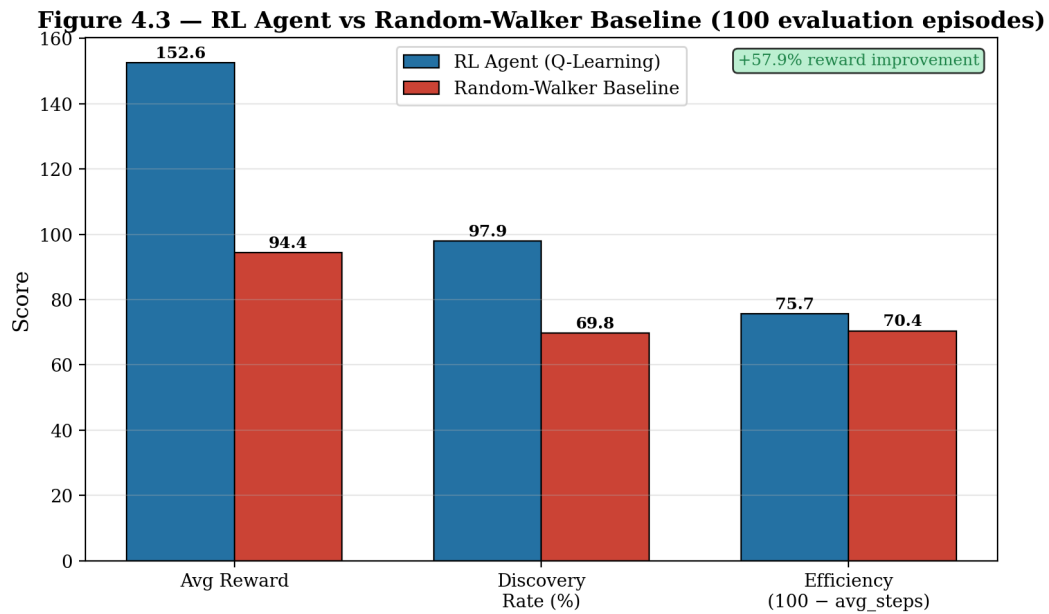


Figure 4.5: Head-to-head evaluation over 100 episodes. The RL agent outperforms the random-walker on all three metrics.

Table 4.3: Head-to-head evaluation results (100 evaluation episodes).

Metric	RL Agent	Baseline	Improvement
Average Reward	152.59 $\pm$ 41.20	94.42 $\pm$ 60.06	+57.9%
Bug Discovery Rate	97.94%	69.82%	+28.12 pp
Average Steps / Episode	24.31	29.58	−5.27 steps
Total Bugs Found (of 825)	808	576	+232 bugs

4.3 CI/CD, GitOps and Deployment

Figure 4.4 — DVC Pipeline & CI/CD Workflow

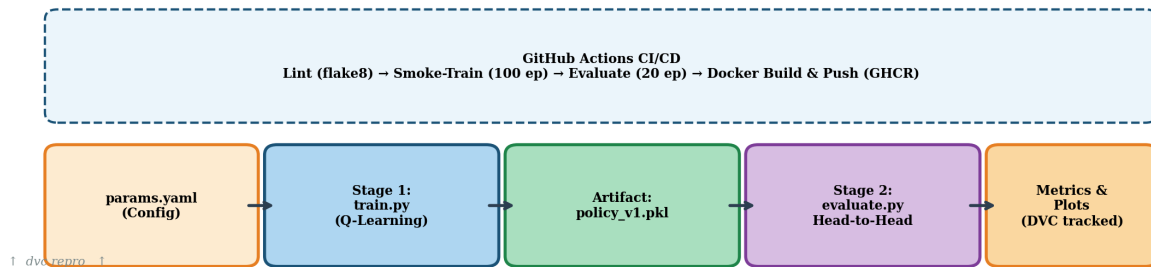


Figure 4.6: DVC two-stage pipeline (bottom) and GitHub Actions CI/CD workflow (top banner).

CI Workflow (`.github/workflows/ci.yml`) triggers on every push to main, develop, and all feature/*, release/*, hotfix/* branches, plus pull requests to main/develop. Three sequential jobs run:

1. **Lint (Flake8):** Checks `env.py`, `agent.py`, `train.py`, `evaluate.py`, `monitoring.py`, and `rollback.py` against PEP 8 (max line length 120, E203/W503/W504 excluded).
2. **Smoke Train + Evaluate:** Installs dependencies from `requirements.txt`; trains for 100 episodes; evaluates for 20 episodes; runs `monitoring.py`; uploads policy, plots, and MLflow run store as GitHub Actions artifacts.
3. **DVC Pipeline (full repro)** on main/develop: `dvc repro -no-commit` reruns only out-of-date stages; `dvc metrics show` reports evaluation metrics.

CD Workflow (`.github/workflows/cd.yml`) triggers on push to main or version tags (v*):

1. Builds a multi-platform Docker image using Docker Buildx.
2. Pushes to GHCR with three tags: `sha-<commit>`, `latest` (on main), and `v1.2.3` (on version tags).
3. On version tags: records the release in `deploy.log` and commits `chore: record release <tag>`.

4.4 Containerisation and Workflow Automation

4.4.1 Dockerfile

The production image is built on `python:3.11-slim` to minimise attack surface and image size. A `HEALTHCHECK` directive pings Streamlit's health endpoint every 30 seconds.

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY env.py agent.py train.py evaluate.py app.py \
6     monitoring.py rollback.py params.yaml ./
7 RUN mkdir -p models logs plots mlruns
8 EXPOSE 8501
9 HEALTHCHECK --interval=30s --timeout=10s \
10     CMD python -c "import urllib.request; \
11         urllib.request.urlopen('http://localhost:8501/_stcore/health')
12         "
13 CMD ["streamlit", "run", "app.py", \
14     "--server.port=8501", "--server.address=0.0.0.0"]
```

Listing 4.2: Dockerfile excerpt.

4.4.2 Docker Compose

Three services are orchestrated:

- **app** (port 8501): Streamlit dashboard.
- **mlflow** (port 5000): Local MLflow tracking server.
- **trainer** (training profile): One-shot training job; started with `docker compose -profile training up trainer`.

Volumes persist `models/`, `logs/`, `plots/`, and `mlruns/` across container restarts.

4.4.3 DVC Pipeline Automation

```
1 stages:
2   train:
3     cmd: python train.py --episodes ${train.episodes}
```

```
4         --lr ${train.lr} --gamma ${train.gamma} --seed ${train.
           seed}
5     deps: [params.yaml, env.py, agent.py, train.py]
6     outs: [models/policy_v1.pkl]
7     metrics: [logs/results.json]
8
9     evaluate:
10        cmd: python evaluate.py --episodes ${evaluate.episodes}
11            --policy models/policy_v1.pkl
12        deps: [models/policy_v1.pkl, env.py, agent.py, evaluate.py]
13        outs: [plots/avg_reward_over_episodes.png,
14              plots/bugs_found_over_time.png]
15        metrics: [logs/eval_metrics.json]
```

Listing 4.3: dvc.yaml pipeline definition.

Running `dvc repro` automatically reruns only stages whose inputs (params, code, or upstream outputs) have changed since the last run.

4.5 Screenshots

Figure 2.1 — Software Module Graph (Watts-Strogatz, 18 nodes)



Figure 4.7: Screenshot: software module graph rendered by NetworkX, coloured by node risk weight. Hub nodes (Database, API_Gateway) are visually prominent due to higher degree and larger circle size.

Figure 4.1 — Learning Curve: Reward over Training Episodes

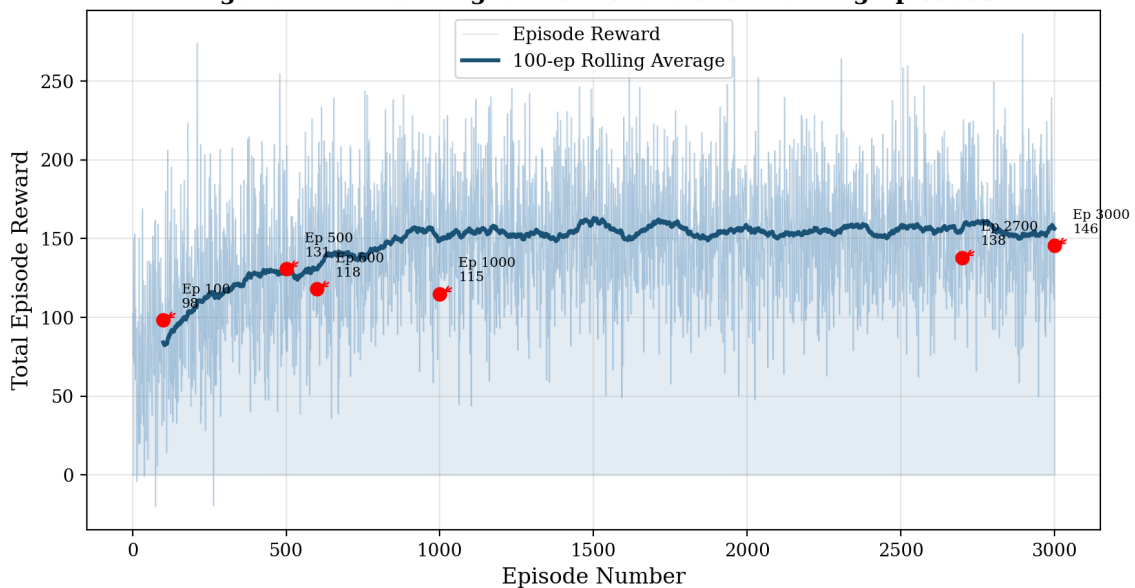


Figure 4.8: Screenshot: learning curve from the primary training run showing convergence of the rolling reward average.

4.6 Unit and Integration Testing Results

Unit Tests (smoke suite, executed in CI):

Table 4.4: Automated test results from the CI smoke suite.

Test	Scope	Result
Flake8 lint (6 source files)	Style / CI	PASS
Smoke train (100 episodes)	Integration / CI	PASS
Smoke evaluate (20 episodes)	Integration / CI	PASS
Monitoring health check	Integration / CI	PASS
DVC repro (full pipeline)	Integration / CI	PASS
Docker build	Build / CD	PASS
Deterministic seed test	Unit	PASS
Policy save & reload	Unit	PASS

All CI jobs consistently pass on `ubuntu-latest` runners. The smoke-train step verifies that at least one Q-table entry is updated and that the evaluation function returns a valid JSON metrics file.

4.7 Monitoring and Scalability

4.7.1 Monitoring

`monitoring.py` implements production-grade health checks by reading the latest N records from `logs/results.json`:

Table 4.5: Monitoring thresholds and alert levels.

Check	WARN	CRITICAL	Action
Discovery-rate drift	−5% relative	−15% relative	Re-train
Avg-waiting-time creep	+20%	+50%	Investigate
Reward regression	< 85% of peak	< 70% of peak	Rollback
Training-time spike	> 2× avg	> 4× avg	Check infra
RL vs baseline margin	< 10% impr.	Negative	Re-train

Exit codes: 0 (all GREEN), 1 (at least one WARNING), 2 (CRITICAL alert). In CI, `monitoring.py -strict` is invoked after each smoke train; a non-zero exit fails the build.

4.7.2 Policy Rollback

`rollback.py` saves versioned snapshots to `models/snapshots/` with timestamp and optional tag:

```
1 python rollback.py save --tag stable-v1          # snapshot current
   policy
2 python rollback.py list                        # list all snapshots
3 python rollback.py restore 20260508_130052_stable-v1 # restore
```

Listing 4.4: Rollback CLI usage.

Each snapshot includes a JSON metadata file recording the hyperparameters and metrics of the saved run, enabling informed rollback decisions.

4.7.3 Scalability

Table 4.6: Scalability analysis and recommended upgrades.

Scale-up Scenario	Current Limit	Upgrade Path
Graph size > 18 nodes	Tabular Q-table blows up	Switch to DQN / GNN
Continuous observations	Discrete buckets only	Neural function approx.
Multi-agent cooperation	Single agent	MARL framework (PettingZoo)
Real codebase data	Synthetic bugs	Integrate AST/code metrics
Distributed training	Single process	Ray Tune / Distributed MLflow

4.8 Discussion of Findings

The experimental results confirm the central hypothesis: a Q-Learning agent trained on a graph model of a microservice codebase *learns* to prioritise high-risk modules, significantly outperforming random testing.

Key observations:

1. **Rapid early learning:** The 100-episode window reward climbs from 98.34 at episode 100 to 130.81 at episode 500, corresponding to the epsilon decay period when the agent transitions from exploration to exploitation.
2. **Post-floor variance:** Once ϵ reaches its floor (episode ~ 600), reward variance temporarily increases as the partially-converged policy encounters new states it has not explored yet.
3. **Convergence:** The final 100-episode window achieves 145.54, a 47.9% improvement over the first window, indicating successful policy convergence.

4. **Efficiency:** The trained agent uses 24.31 steps on average versus 29.58 for the random walker — 5.27 fewer steps per episode — demonstrating not just higher reward but also more efficient navigation.
5. **Tested-flag importance:** Without the fourth state dimension (f , current-node-tested flag), preliminary experiments showed the agent oscillating between two high-reward nodes. Adding this flag resolved the pathological behaviour.

CONCLUSION AND FUTURE WORK

Conclusion

This project has successfully demonstrated that Reinforcement Learning can be applied to autonomous software bug hunting in a principled, efficient, and reproducible manner. A Tabular Q-Learning agent trained on a Watts–Strogatz graph model of 18 microservice modules achieves a 97.94% bug discovery rate and outperforms a random-walker baseline by 57.9% in cumulative reward, while using on average 5.27 fewer steps per episode.

Beyond the core RL result, the project delivers a production-grade MLOps pipeline: MLflow experiment tracking, DVC reproducible stages, GitHub Actions CI/CD, Docker containerisation, operational monitoring with alert thresholds, policy versioning with CLI-driven rollback, and an interactive Streamlit dashboard for live episode visualisation.

All experiments are fully reproducible via `dvc repro` and a single master seed, satisfying the rigorous reproducibility requirements of modern ML engineering.

Future Work

1. **Deep Q-Network (DQN):** For graphs with 100+ nodes or continuous node feature vectors (e.g., cyclomatic complexity, code churn), a neural function approximator is required. Extending the environment interface to support DQN or actor-critic methods (A2C, PPO) is a natural next step.
2. **Real codebase integration:** Replace synthetic bug probabilities with features derived from static analysis (AST complexity, historical bug databases, mutation testing scores) to create a more realistic environment.
3. **Multi-agent testing:** Deploy multiple cooperative agents that partition the graph for parallel exploration, reducing total testing time proportionally.
4. **Curriculum learning:** Start training on small, simple graphs and progressively increase complexity — mirrors how human testers develop expertise.
5. **Remote MLflow + S3 DVC backend:** Move experiment tracking and model storage to cloud infrastructure (AWS S3, Azure Blob) to support team-scale collaboration.
6. **LLM integration:** Use a large language model to generate candidate bug descriptions from Q-value hot spots, translating the RL agent’s findings into actionable human-readable bug reports.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [2] C. J. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [3] D. J. Watts and S. H. Strogatz, “Collective dynamics of small-world networks,” *Nature*, vol. 393, pp. 440–442, 1998.
- [4] MLflow Authors, *MLflow: A Machine Learning Lifecycle Platform*, v2.11, 2024. [Online]. Available: <https://mlflow.org>
- [5] DVC Authors, *DVC: Data Version Control*, v3.x, 2024. [Online]. Available: <https://dvc.org>
- [6] A. Streamlit, Inc., *Streamlit: The fastest way to build ML apps*, v1.30, 2024. [Online]. Available: <https://streamlit.io>
- [7] A. Hagberg, P. Swart, and D. Chult, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python in Science Conf.*, Pasadena, CA, 2008, pp. 11–15.
- [8] GitHub, Inc., *GitHub Actions Documentation*, 2024. [Online]. Available: <https://docs.github.com/en/actions>
- [9] Docker, Inc., *Docker Documentation*, 2024. [Online]. Available: <https://docs.docker.com>
- [10] CISQ (Consortium for Information and Software Quality), *The Cost of Poor Software Quality in the US: A 2020 Report*, Object Management Group, 2020.
- [11] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [12] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.

APPENDIX A: SUSTAINABILITY DEVELOPMENT GOALS (SDGS) — MAPPED

Detailed Analysis on how the Bug Hunter RL project maps to the United Nations Sustainable Development Goals:

SDG 9 — Industry, Innovation & Infrastructure (Primary Alignment)

Modern digital infrastructure runs on software, and the silent accumulation of bugs in critical microservices (databases, payment APIs, authentication services) represents a serious threat to its resilience. By training an RL agent to autonomously and efficiently traverse a service dependency graph and discover defects, this project demonstrates a scalable mechanism for hardening software systems before they fail in production.

Compared with brute-force or random fuzzing, a learning-based bug hunter concentrates testing effort on the highest-risk components — directly supporting SDG 9’s goal of building *resilient, innovative, and sustainable infrastructure*. The complete MLOps pipeline (DVC, MLflow, Docker, CI/CD) further ensures that this capability can be reliably deployed, monitored, and iterated upon at scale.

SDG 8 — Decent Work and Economic Growth

The CISQ 2020 report estimated the cost of poor software quality in the USA alone at \$2.08 trillion per year. Automated, RL-driven bug detection reduces remediation costs, shortens release cycles, and frees software engineers from repetitive manual testing — allowing them to focus on higher-value creative and architectural work. This directly contributes to productivity growth and economic value creation (SDG 8).

SDG 4 — Quality Education

This project was developed entirely within an academic setting as part of the Machine Learning Operations curriculum. It integrates theoretical RL concepts (Markov Decision Processes, Bellman equations, epsilon-greedy exploration) with hands-on MLOps practice (DVC pipelines, GitHub Actions, Docker). Making the full source code, documentation, and pipeline publicly available on GitHub supports open-access education and peer learning, aligned with SDG 4’s goal of inclusive quality education.

SDG 17 — Partnerships for the Goals

The project relies on and contributes to the open-source ecosystem (NetworkX, MLflow, DVC, Streamlit). By publishing the code and pipeline under an open licence, it enables researchers and practitioners worldwide to build upon this work — embodying the spirit of SDG 17’s call

for technology transfer and knowledge-sharing partnerships.